Maparea obiectual relaţională

Seminar 2

Cuprins

- Fundamentele ORM
- Hibernate (Java) și Entity Framework Core (C#)
- Maparea entităților și a relațiilor
- Limbajele de interogare ORM (JPQL/HQL și LINQ)
- Partajarea conexiunilor (connection pooling)
- HikariCP și .NET
- Măsurarea performanței și comparație
- Migrarea de la JDBC/ADO.NET la ORM

Ce este ORM?

The Problem:

- Programarea OO folosește obiecte
- BD relaționale folosesc tabele și înregistrări
- Nepotrivire: obiecte ≠ data relaționale

Soluția ORM:

- Mapare automată între obiecte și tabele din BD
- Conversia operațiilor pe obiecte în SQL
- Gestionarea relațiilor, încărcare întârziată/imediată
- Reducerea semnificativă a codului "de umplutură" (boilerplate)

ORM-uri populare:

- Java: Hibernate, EclipseLink, OpenJPA
- .NET: Entity Framework Core, NHibernate, Dapper

Beneficiile și dezavantajele utilizării ORM

Beneficii:

- Mai puțin cod duplicate (boilerplate) - fără procesarea manuală a ResultSet
- Interogări type-safe
- Generate automată de SQL
- Independență de DB (mostly)
- Caching implementat intern
- Gestionarea relațiilor
- Gestionarea tranzacțiilor

Dezavantaje:

- Curba de învățare și complexitatea
- Reducerea performanței (câteodată)
- Mai puțin control la generarea codului SQL
- Poate genera interogări SQL ineficiente dacă nu este folosit corespunzător
- Depanarea poate fi mai dificilă
- Poate ascunde probleme (N+1 queries)

Regula de bază:

- Folosim ORM pentru operații tipice CRUD și luăm în considerare folosirea SQL-ul nativ pentru interogări complexe sau pentru operații critice dpdv al performanței.

Hibernate - prezentare generală

Ce este Hibernate:

- Cel mai popular framework Java pentru ORM
- Implementează specificația JPA (Java Persistence Architecture)
- Putem utiliza adnotări JPA (standard) sau specifice Hibernate

Componente de bază:

- **SessionFactory**: Obiect factory, thread-safe, pentru **Sessions** (unul pe aplicație)
- **Session**: Reprezintă o conexiune/tranzacție la baza de date
- **EntityManager**: Interfața JPA (preferată dpdv al portabilității)
- **Transaction**: Gestionează tranzacțiile DB
- **Query/Criteria**: Folosite pentru efectuarea interogărilor

Configurarea

- persistence.xml (standardul JPA)
- Hibernate.cfg.xml (specific Hibernate)
- Configurare programatică

Entity Framework Core - scurtă prezentare

Ce este EF Core:

- ORM modern de la Microsoft pentru .NET
- Cross-platform (Linux, macOS, Windows)
- Abordare code-first sau DB-first

Componente de bază:

- **DbContext**: Reprezintă o conexiune/tranzacție la baza de date
- **DbSet**: Reprezintă o tabelă în baza de date
- **ChangeTracker**: Monitorizează schimbările entităților
- **Migration**: Versionarea schemei și actualizări

Configurarea

- Fluent API
- Adnotări (atribute)
- appsettings.json pentru BD

Configurare Hibernate – Maven (pom.xml)

```
<dependencies>

 <!-- Hibernate Core -->
 <dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.0.Final</version>
 </dependency>

 <!-- PostgreSQL Core -->
 <dependency>…</dependency>

 <!-- HikariCP (Connection Pool) -->
 <dependency>…</dependency>

</dependencies>
```

Configurare Hibernate – persistence.xml
src/main/resources/META-INF/persistence.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0">
<persistence-unit name="myPersistenceUnit">
  <!-- Entity classes -->
  <class>com.example.model.Department</class>
  <class>com.example.model.Employee</class>
  <properties>
    <!-- Database connection -->
    <property name="jakarta.persistence.jdbc.url" value="jdbc:postgresql://localhost:5432/mydb"/>
    <property name="jakarta.persistence.jdbc.user" value="postgres"/>
    <property name="jakarta.persistence.jdbc.password" value="password"/>
    <property name="jakarta.persistence.jdbc.driver" value="org.postgresql.Driver"/>
    <!-- Hibernate settings -->
    <property name="hibernate.dialect" value="org.hibernate.dialect.PostgreSQLDialect"/>
    <property name="hibernate.show_sql" value="true"/>
    <property name="hibernate.format_sql" value="true"/>
    <property name="hibernate.hbm2ddl.auto" value="update"/>
  </properties>
</persistence-unit></persistence>
```

Configurare EF Core

 # .NET CLI

```
# EF Core with PostgreSQL provider
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL

# EF Core Design (for migrations)
dotnet add package Microsoft.EntityFrameworkCore.Design

# Optional: EF Core Tools
dotnet tool install --global dotnet-ef
```

 # Package Manager Console (Visual Studio):

```
Install-Package Npgsql.EntityFrameworkCore.PostgreSQL
Install-Package Microsoft.EntityFrameworkCore.Design
```

Configurare EF Core - DbContext

```
using Microsoft.EntityFrameworkCore;
public class ApplicationDbContext : DbContext {
   public DbSet<Department> Departments { get; set; }
   public DbSet<Employee> Employees { get; set; }
   // Constructor for configuration injection
   public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
   // Alternative: configure directly in DbContext
   protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) {
      if (!optionsBuilder.IsConfigured) { optionsBuilder.UseNpgsql(
"Host=localhost;Database=mydb;Username=postgres;Password=password;" + "Maximum Pool        Size=10;Minimum Pool
Size=5");
         optionsBuilder.UseLazyLoadingProxies();  // Enable lazy loading (optional)
         optionsBuilder.LogTo(Console.WriteLine); // Enable SQL logging
}}
   // Configure entity relationships using Fluent API
   protected override void OnModelCreating(ModelBuilder modelBuilder) {
   modelBuilder.Entity<Department>()
       .HasMany(d => d.Employees)
      .WithOne(e => e.Department)
      .HasForeignKey(e => e.DepartmentId); } }
```

Crearea claselor **entități** - Hibernate/JPA 1/2

```
import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "departments")
public class Department {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   @Column(name = "name", nullable = false, length = 100)
   private String name;
   @Column(name = "location", length = 100)
   private String location;
   @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
   private List<Employee> employees;

   public Department() {}
   public Department(String name, String location) {this.name = name; this.location = location; }
   public Long getId() { return id; }
   public void setId(Long id) { this.id = id; }
   // … all the remaining setters & getters
}
```

Crearea claselor **entități** - Hibernate/JPA 2/2

```
import jakarta.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "employees")
public class Employee {
   @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;
   @Column(name = "name", nullable = false, length = 100)
   private String name;
   @Column(name = "email", length = 100)
   private String email;
   @Column(name = "salary", precision = 10, scale = 2)
   private BigDecimal salary;
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "department_id")
   private Department department;
   public Employee() {}
   public Employee(String name, String email, BigDecimal salary) {
      this.name = name; this.email = email; this.salary = salary; }
   // Getters and Setters.....
}
```

Crearea claselor **entități** - EF Core 1/2

```
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("departments")]
public class Department {
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [Required]
    [MaxLength(100)] [Column("name")]
    public string Name { get; set; }

    [MaxLength(100)] [Column("location")]
    public string Location { get; set; }

    // Navigation property for the one-to-many relationship
    public virtual ICollection<Employee> Employees { get; set; }
    public Department() {
        Employees = new List<Employee>();
    }
}
```

Crearea claselor **entități** - EF Core 2/2

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
[Table("employees")]
public class Employee {
   [Key]
   [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
   public int Id { get; set; }

   [Required] [MaxLength(100)] [Column("name")]
   public string Name { get; set; }

   [MaxLength(100)] [Column("email")]
   public string Email { get; set; }

   [Column("salary", TypeName = "decimal(10,2)")]
   public decimal? Salary { get; set; }

   [Column("department_id")]
   public int DepartmentId { get; set; }
   // Navigation property [ForeignKey("DepartmentId")]
   public virtual Department Department { get; set; }
}
```

Adnotări JPA

La nivel de clasa:

- @Entity - marchează o clasă ca fiind o entitate JPA
- @Table(name="nume tabelă") - leagă entitatea de o tabela DB specifică

La nivel de câmp din obiect:

- @Id - marchează câmpul folosit ca și cheie primară
- @GeneratedValue (stragegy=…) - generarea valorii cheii primare
- @Column (name="col_name") - mapează câmpul la o coloană specifică
- @Column (nullable=false) - adaugă constrângerea NOT NULL la coloana asociată
- @Column (length=100) - fixează lungimea maximă a coloanei

Adnotări pentru relații:

- @OneToMany(mappedBy="field") - Relație one-to-many
- @ManyToOne - Relație many-to-one
- @JoinColumn(name="fk_column") - coloana cheie străină
- fetch = FetchType.LAZY - implicit pentru @OneToMany
- fetch = FetchType.EAGER
- cascade = CascadeType.ALL

Adnotări EF Core

La nivel de clasa:

- [Table("nume tabelă")] - leagă entitatea de o tabela DB specifică

La nivel de câmp din obiect:

- [Key] - marchează câmpul folosit ca și cheie primară
- [DatabaseGenerated(DatabaseGeneratedOption.Identity)] - generarea valorii cheii primare (autogenerare)
- [Column("col_name")] - mapează câmpul la o coloană specifică
- [Required] - adaugă constrângerea NOT NULL la coloana asociată
- [MaxLength(100)] - fixează lungimea maximă a coloanei
- [ForeignKey("PropertyName")] - Relație cheie străină

Adnotări pentru relații/navigare:

- **virtual** - Activează încărcarea întârziată (necesită proxy)
- **ICollection** – Proprtietate pentru navigarea colecției

Inițializarea EntityManagerFactory - Java

```java
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
public class JPAUtil {
   private static EntityManagerFactory emf;
   public static EntityManagerFactory getEntityManagerFactory() {
      if (emf == null) {
         emf = Persistence.createEntityManagerFactory("myPersistenceUnit");
      } return emf;
   }
   public static void closeEntityManagerFactory() {
      if (emf != null && emf.isOpen()) { emf.close(); }
   }
}
```

Usage:

```java
EntityManagerFactory emf = JPAUtil.getEntityManagerFactory();
EntityManager em = emf.createEntityManager();
// ... use EntityManager
em.close();
```

Note: EntityManagerFactory is thread-safe and expensive to create. Create once per application.

Inițializarea DbContext – EF core

```
// Option 1: Direct instantiation
using (var context = new ApplicationDbContext()) {
   // Use context
}
// Option 2: With options (dependency injection)
var optionsBuilder = new DbContextOptionsBuilder<ApplicationDbContext>();
optionsBuilder.UseNpgsql(connectionString);
using (var context = new ApplicationDbContext(optionsBuilder.Options)) {
// Use context
}
```

Best Practice - Singleton Pattern for Configuration:

```
public class DbContextFactory {
   private static string _connectionString;
   public static void Initialize(string connectionString) {
      _connectionString = connectionString;
   }
   public static ApplicationDbContext CreateContext() {
      var optionsBuilder = new DbContextOptionsBuilder<ApplicationDbContext>();
      optionsBuilder.UseNpgsql(_connectionString);
      return new ApplicationDbContext(optionsBuilder.Options);
   }
}
```

Operațiuni de bază CRUD – Hibernate 1/2

Create (Insert):

```
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();
try {
   tx.begin();
   Department dept = new Department("Engineering", "Building A");
   em.persist(dept);
   tx.commit();
} catch (Exception e) {
   if (tx != null && tx.isActive()) {
      tx.rollback();
   } e.printStackTrace();
} finally { em.close(); }
```

Read (Find by ID):

```
EntityManager em = emf.createEntityManager();
try {
   Department dept = em.find(Department.class, 1L);
   if (dept != null) {
      System.out.println(dept.getName());
   }
} finally { em.close(); }
```

Operațiuni de bază CRUD – Hibernate 2/2

Update:

```
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();
try {
   tx.begin();
   Department dept = em.find(Department.class, 1L);
   dept.setLocation("Building B");
   tx.commit();
} catch (Exception e) {
   if (tx != null && tx.isActive()) {
      tx.rollback();
   } e.printStackTrace();
} finally { em.close(); }
```

Delete

```
EntityManager em = emf.createEntityManager(); EntityTransaction tx = em.getTransaction();
try {
   Department dept = em.find(Department.class, 1L);
   if (dept != null) {em.remove(dept); }
   tx.commit();
} catch (Exception e) { … } finally { em.close(); }
```

Operațiuni de bază CRUD – EFCore 1/2

Create (Insert):

```
using (var context = new ApplicationDbContext()) {
    var dept = new Department { Name = "Engineering", Location = "Building A" };
    context.Departments.Add(dept);
    context.SaveChanges();
}
```

Read (Find by ID):

```
using (var context = new ApplicationDbContext()) {
    var dept = context.Departments.Find(1);
    if (dept != null) {
        Console.WriteLine(dept.Name);
    }
}
```

Operațiuni de bază CRUD – EFCore 2/2

Update:

```
using (var context = new ApplicationDbContext()) {
   var dept = context.Departments.Find(1);
   if (dept != null) {
      dept.Location = "Building B";
      context.SaveChanges(); // Automatically detects changes
   }
}
```

Delete:

```
using (var context = new ApplicationDbContext()) {
   var dept = context.Departments.Find(1);
   if (dept != null) {
      context.Departments.Remove(dept);
      context.SaveChanges();
   }
}
```

Interogări JPQL (Java Persistence Query Language) - 1/2

Get All Departments:

```
EntityManager em = emf.createEntityManager();
try {
   List<Department> departments =
      em.createQuery( "SELECT d FROM Department d", Department.class) .getResultList();
   for (Department dept : departments) {
      System.out.println(dept.getName());
   }
} finally { em.close(); }
```

Interogare cu parametrii:

```
List<Employee> employees
   = em.createQuery( "SELECT e FROM Employee e WHERE e.department.id = :deptId AND e.salary > :minSalary",
Employee.class)
   .setParameter("deptId", 1L)
   .setParameter("minSalary", new BigDecimal("50000"))
   .getResultList();
```

Interogări JPQL (Java Persistence Query Language) 2/2

Join Fetch (Eager Loading):

```
List<Department> departments =
   em.createQuery( "SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees",
Department.class) .getResultList();
// Now employees are loaded, no additional query needed
for (Department dept : departments) {
   for (Employee emp : dept.getEmployees()) {
      System.out.println(emp.getName());
   }
}
```

Interogări EFCore – LINQ (Language Integrated Query)

Get All Departments:

```
using (var context = new ApplicationDbContext()) {
   var departments = context.Departments.ToList();
   foreach (var dept in departments) {
      Console.WriteLine(dept.Name);
   }
}
```

Interogare cu paremetrii:

```
using (var context = new ApplicationDbContext()) {
   var employees = context.Employees
      .Where(e => e.DepartmentId == 1 && e.Salary > 50000)
      .ToList();
}
```

Eager Loading with Include:

```
using (var context = new ApplicationDbContext()) {
   var departments = context.Departments .Include(d => d.Employees) .ToList();
   // Employees are loaded, no additional query
   foreach (var dept in departments) {
      foreach (var emp in dept.Employees) {
         Console.WriteLine(emp.Name);
      }
   }
}
```

Încărcare întârziată (lazy) vs încarcare imediată (eager)

Încărcare întârziată (implicit pentru @OneToMany în Hibernate):

```
Department dept = em.find(Department.class, 1L);
// Employees NOT loaded yet
List<Employee> employees = dept.getEmployees();
// NOW employees are loaded (separate SQL query)
```

Problema: Paote cauză N+1 queries:

```
List<Department> depts = em.createQuery("SELECT d FROM Department d", Department.class) .getResultList();
// 1 query
for (Department dept : depts) {
   System.out.println(dept.getEmployees().size());
   // N queries!
}
```

Soluția: Încărcare imediată cu JOIN FETCH:

```
List<Department> depts =
   em.createQuery( "SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees", Department.class)
.getResultList();
// Only 1 query!
for (Department dept : depts) {
   System.out.println(dept.getEmployees().size()); // No additional query
}
```

Încărcarea întârziată - EFCore

Option 1: Using Proxies (Requires virtual keyword):

```
// Install package
dotnet add package Microsoft.EntityFrameworkCore.Proxies
// Enable in DbContext
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) {
   optionsBuilder .UseNpgsql(connectionString) .UseLazyLoadingProxies();
}
// Usage
var dept = context.Departments.Find(1);
// Employees loaded automatically when accessed
var employees = dept.Employees;
```

Option 2: Explicit Loading:

```
var dept = context.Departments.Find(1);
// Explicitly load employees
context.Entry(dept).Collection(d => d.Employees).Load();
```

Option 3: Eager Loading (Recommended):

```
var dept = context.Departments .Include(d => d.Employees) .FirstOrDefault(d => d.Id == 1)
```

Comparație JDBC vs Hibernate Code

JDBC Code (lab1):

```
String sql = "SELECT * FROM employees WHERE department_id = ?";
try (Connection conn = DriverManager.getConnection(url, user, pass);
   PreparedStatement pstmt = conn.prepareStatement(sql)) {
   pstmt.setInt(1, departmentId);
   try (ResultSet rs = pstmt.executeQuery()) {
      List<Employee> employees = new ArrayList<>();
      while (rs.next()) {
         Employee emp = new Employee();
         emp.setId(rs.getLong("id"));
         emp.setName(rs.getString("name"));
         emp.setEmail(rs.getString("email"));
         emp.setSalary(rs.getBigDecimal("salary"));
         employees.add(emp);
      }
      return employees; } }
```

Hibernate Code (Lab 2):

```
EntityManager em = emf.createEntityManager();
try {
   return em.createQuery( "SELECT e FROM Employee e WHERE e.department.id = :deptId",
      Employee.class) .setParameter("deptId", departmentId) .getResultList();
} finally { em.close(); }
```

Comparație ADO.NET vs EFCore Code

ADO.NET Code (Lab 1):

```
string sql = "SELECT * FROM employees WHERE department_id = @deptId";
var employees = new List<Employee>();
using (var conn = new NpgsqlConnection(connectionString)) {
   conn.Open();
   using (var cmd = new NpgsqlCommand(sql, conn)) {
      cmd.Parameters.AddWithValue("@deptId", departmentId);
      using (var reader = cmd.ExecuteReader()) {
         while (reader.Read()) {
            var emp = new Employee {
               Id = reader.GetInt32(0),
               Name = reader.GetString(1),
               Email = reader.GetString(2),
               Salary = reader.GetDecimal(3) };
            employees.Add(emp);
         } } } }
return employees;
```

EF Core Code (Lab 2):

```
using (var context = new ApplicationDbContext()) {
   return context.Employees .Where(e => e.DepartmentId == departmentId) .ToList();
}
```

Connection pool (Grup de conexiuni)

La ce folosește un connection pool

Fără connection pool:

- **Request 1: Create conn -> Close -> Destroy**
- **Request 2: Create conn -> Close -> Destroy**
- **Request 3: Create conn -> Close -> Destroy**
- **Fiecare creare de conexiunea ia timp (~50..100 ms)**

Folosind un connection pool:

- **Request 1: Get from pool -> Use -> Return to pool**
- **Request 2: Get from pool -> Use -> Return to pool**
- **Request 3: Get from pool -> Use -> Return to pool**
- **Conexiunile la DB sunt reutilizate (< 1 ms)**

Beneficii:
Obținerea unei conexiuni e mult mai rapidă

- **Reduce încărcarea serverului BD**
- **O mai bună utilizare a resurselor**
- **Limite configurate previne epuizarea resurselor**

Connection pool Cum funcționează

Ciclul de viață:

- **Inițializarea:** Grupul crează un număr minim de conexiuni
- **Request:** Aplicația cere o conexiune
  - Conexiune disponibilă: Întoarce o conexiune existentă
  - Nu există conexiune disponibilă, dar sub max: Crează conexiune nouă
  - Dacă suntem la max: Așteaptă eliberare conexiune sau aruncă excepție
- **Utilizare:** Aplicația folosește conexiune
- **Returnare:** Aplicația închide conexiune -> se întoarce în grup (nu este închisă efectiv)
- **Mentenanța:** Pool-ul validează și menține conexiunile
- **Inchidere:** Pool-ul închide toate conexiunile

Parametrii cheie:

- **Minimum Pool Size: Numărul minim de conexiune menținute deschise când nu există activitate**
- **Maximum Pool Size: Numărul maxim de conexiuni deschise/menținute**
- **Connection Timeout: Timpul maxim de așteptare pentru o conexiune**
- **Max Lifetime: Timpul maxim de utilizare a unei conexiuni până la reînnoire**

HikariCP - Java Connection Pool

De ce să folosim HikariCP ?

- Un connection pool foarte rapid pentru Java
- Implicit in Spring Boot
- Configurare simplă
- Performanță excelentă

Pentru integrarea HikariCP in persistence.xml - va fi folosit automat de către Hiberante pentru toate conexiunile la DB:

```
<property name="hibernate.connection.provider_class"
value="org.hibernate.hikaricp.internal.HikariCPConnectionProvider"/>
<!-- HikariCP settings -->
<property name="hibernate.hikari.maximumPoolSize" value="10"/>
<property name="hibernate.hikari.minimumIdle" value="5"/>
<property name="hibernate.hikari.connectionTimeout" value="30000"/>
<property name="hibernate.hikari.idleTimeout" value="600000"/>
<property name="hibernate.hikari.maxLifetime" value="1800000"/>
<property name="hibernate.hikari.autoCommit" value="true"/> <property
name="hibernate.hikari.connectionTestQuery" value="SELECT 1"/>
```

Connection Pool în .NET

.NET are un mecanism de connection pool inclus în mod implicit

Cum funcționează:

- Se bazează pe potrivirea între connection string
- Conexiunile sunt returnate în pool când sunt distruse (disposed)
- Pool/ul gestionează automat ciclul de viață al conexiunilor

```
// configuration
string connectionString = "Host=localhost;" + "Database=mydb;" + "Username=postgres;" + "Password=password;" +
"Maximum Pool Size=10;" + "Minimum Pool Size=5;" + "Connection Idle Lifetime=300;" + "Connection Pruning
Interval=10;" + "Timeout=30";

// usage

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) { optionsBuilder.UseNpgsql(
"Host=localhost;Database=mydb;..." + "Maximum Pool Size=10;Minimum Pool Size=5");}
```

Resurse și referințe

- **Documentație JDBC:**
  - Official Java Tutorial: https://docs.oracle.com/javase/tutorial/jdbc/
  - PostgreSQL JDBC Driver: https://jdbc.postgresql.org/documentation/
  - API Documentation: https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/
- **ADO.NET Documentation:**
  - Microsoft Docs: https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/
  - Npgsql Documentation: https://www.npgsql.org/doc/
  - System.Data Namespace: https://learn.microsoft.com/en-us/dotnet/api/system.data
- **PostgreSQL:**
  - Official Documentation: https://www.postgresql.org/docs/
  - SQL Tutorial: https://www.postgresql.org/docs/current/tutorial.html